University of Essex

FIND OUT MORE

Why stay?

- Gain advanced skills and knowledge that you need for a competitive edge in your career
  - Boost your career prospects!
  - Get support every step of the way and stay connected to our community
- Specialise in your passion
- Switch paths on our conversion Masters
- Be involved in top quality research

[CE310] Evolutionary Computation and Genetic Programming

Riccardo Poli

Genetic Programming

University of Essex
School of Computer Science and Electronic Engineering

What did we learn last week? (1)

- We learned how to deal with **real-valued parameters** in GAs and how to implement the associated **crossover** (interpolation and box) and **mutation** operators.
- **Rank selection** scales the **fitness function** and assigns each individual a rapidly decreasing selection probability.
- **Elitist selection** keeps copies of the N fittest individuals, which are passed on to the next generation.

What did we learn last week? (2)

- We have learned about several cases how **objective functions Q(s)** can be mapped into **fitness functions f(s)**
- Constrained optimisation means minimising $\boldsymbol{U}(\boldsymbol{x})$ subject to the constraints $\boldsymbol{g}_i(\boldsymbol{x}) \geq \mathbf{0}$ (**inequality constraints**) and $\boldsymbol{h}_j(\boldsymbol{x}) = \mathbf{0}$ (**equality constraints**).
- The **penalty method** allows us to transform constrained in unconstrained optimisation problems by adding penalty terms $U'(x) = U(x) + \mathbf{r}_i \cdot \boldsymbol{F}_{\boldsymbol{Penalty}}\big(\boldsymbol{g}_i(\boldsymbol{x})\big) + \cdots$.

What did we learn last week? (3)

- **Evolution strategies** (ESs) typically use arrays of **real numbers** as representation and **mutation** (equal addition of Gaussian random numbers)to explore the search space.
- (Parent + Offspring) vs. (Parents, Offspring) variants
- **Sigma** (step size) Adaptation: 1/5 rule
  - 1/5 offspring better than parents: increase Sigma
  - 1/5 offspring worse than parents: decrease Sigma

Today’s learning objective

- Describe **program representation**, **crossover** and **mutation** operators in Genetic Programming (GP)
- Discuss **terminal** and **function** sets
- Discuss **differences** between **GAs** and **GP** (especially the genetic operators).
- Understand the **range of problems** that GP can solve and how to guide it with **fitness functions**.

Genetic Programming (GP)

- GP is a **systematic**, **domain-independent** method for getting computers to **automatically** solve a problem starting from a **high-level statement** of what needs to be done.
- GP is a special GA where the **individuals** in the population are **computer programs**.
- So, GP **iteratively transforms** a **population** of programs into a new generation of programs under the guidance of a fitness function.
- To do so, GP applies **genetic operations** like recombination, mutation, etc. which are **specialised to handle computer programs**
- Fitness functions are also specialised in many respects.

Executional Steps of GP

1. **Randomly** create an **initial population** of programs from an available set of instructions (**primitives**).
2. **Iterate** the following sub-steps until the termination criterion is satisfied:
   - **Execute** each program and ascertain its fitness.
   - **Select** one or two program(s) from the population based on fitness to participate in genetic operations.
   - **Create** new individual program(s) by applying **genetic operations** with specified probabilities.
3. Return the best-so-far individual

Genetic Operations (1)

- **Cloning**: copy the selected individual program to the new population (generational GP only!).
- **Crossover**: create new offspring program(s) by recombining randomly chosen parts from two selected programs.
- **Mutation**: create one new offspring program by randomly mutating a randomly chosen part of one selected program.
- **Architecture-altering operations**: choose one such operation (e.g., subroutine creation/deletion, dummy argument addition/deletion, …) and create one new offspring using it.

Genetic Operations (2)

- In GAs:

  Generational: **cloning** and **crossover** are **mutually exclusive** (either you create the offspring with one or the other). **Mutation** can be performed on the offspring <u>after</u> such operations or as an <u>independent</u> way of producing offspring.

  Steady state: same but <u>no cloning</u>.

- In GP:
  - *Generational:* **all genetic operations** are **mutually exclusive**. So, either an offspring is created by cloning <u>or</u> by crossover <u>or</u> by mutation <u>or</u> by an architecture altering operation (irrespective of whether GP is generational or steady state).
  - *Steady state*: same but <u>no cloning</u>.

Operator Rates (Probabilities) in GAs

- As a result, **in generational GAs**:

$$p_{XO} + pclone = 1$$

where p_{XO} is the crossover rate and p_{clone} is the cloning rate.

Also, when we talk about mutation rate p_{mut} (or p_m) we normally mean the **probability of hitting any particular bit or gene** with a mutation

Operator Rates (Probabilities) in GP

- Instead, **in generational GP**:

$$p_{cross} + p_{clone} + p_{mut} + parch = 1$$

In GP when we talk about mutation rate p_{mut} we normally mean the **probability of hitting any particular program** with a mutation that can change many instructions at once

In Python…

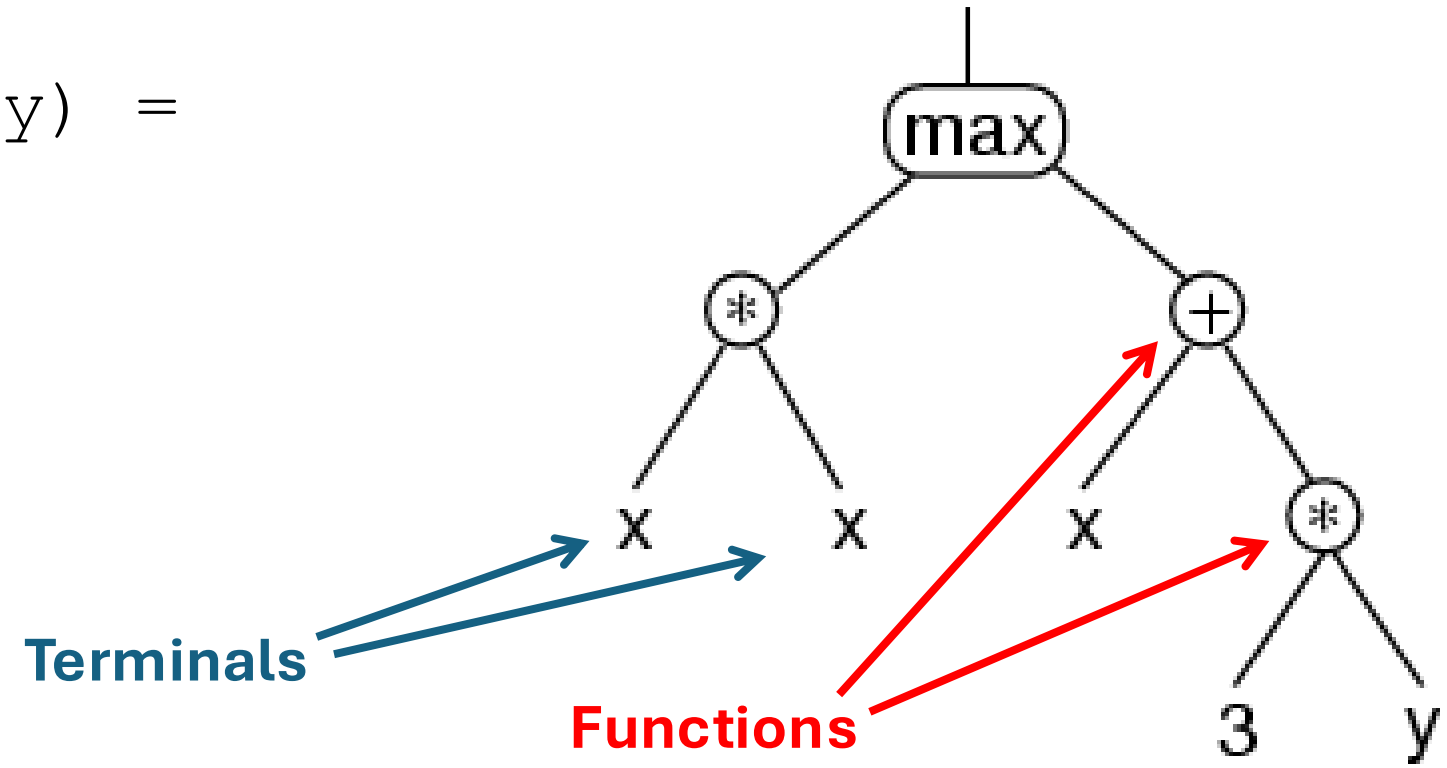
```python
from random import random

p_crossover, p_mutation, p_clone, p_arch_alt = 0.8, 0.1, 0.1, 0.0

random_op_selector = random()

if random_op_selector < p_crossover:
    print('Crossover chosen')
elif random_op_selector < p_mutation + p_crossover:
    print('Mutation chosen')
elif random_op_selector < p_clone + p_mutation + p_crossover :
    print('Cloning chosen')
else:
    print('Architecture altering ops chosen')
```

Program Representation

- Programs are expressed in GP as **syntax trees** rather than as lines of code.

  For example,

  `max(x*x,x+3*y)` =



Random Program Generation

- The programs in the initial population are typically built by **recursively** generating a tree composed of **random choices** of functions and terminals.
- The initial individuals are usually generated subject to a pre-established **maximum size**.

Fitness (1)

- Normally, **fitness evaluation requires executing** the programs in the population **multiple times** within the GP system.
- Nearly always to decide whether a program is *good* or *bad* we need to execute it.
- In some problems we are **interested in the output** produced by a program. Normally, the value returned by the root node is the output of the program while the values of the variables are the inputs.

Fitness (2)

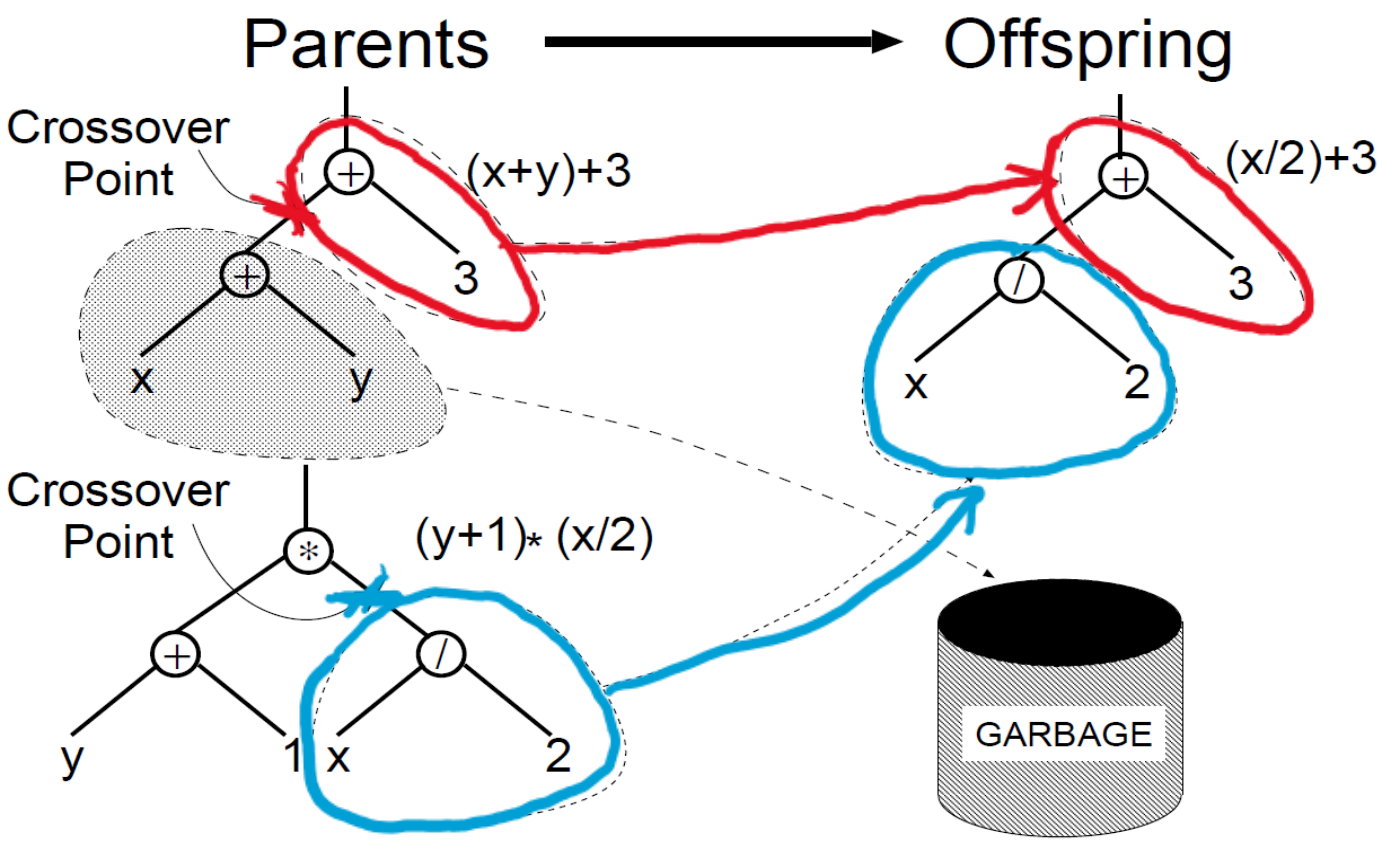
- In other problems we are **interested in the actions** performed by a program. In this case the primitive set will include **functions with side effects** (e.g. changing global data structures or controlling the motors of a robot).
- So, quite often the fitness of a program depends on the results produced by its **execution in many different conditions (fitness cases)**.

Selection

- Genetic operators are applied to individual(s) that are **probabilistically selected** based on fitness.
- Fitter individuals are favoured over less fit individuals.
- The most commonly employed methods for selecting individuals are **tournament** selection and **fitness-proportionate selection**.

Sub-tree Crossover (1)

- Given two parents, **crossover** randomly selects a crossover point in each parent tree and **replaces** the **sub-tree** rooted at one XO point with the sub-tree rooted at the other XO point.



Sub-tree Crossover (2)

- Seen as acting on *expressions*, the same crossover operation corresponds to

  Parent 1: **(x + y) + 3**
  Parent 2: **(y + 1) * (x / 2)**

  Expression excised from parent 1 for deletion: **(x+y)**

  Parent 1 thus donates the structure: **? + 3**

  Expression excised from parent 2 for insertion (replaces **?**): **(x / 2)**

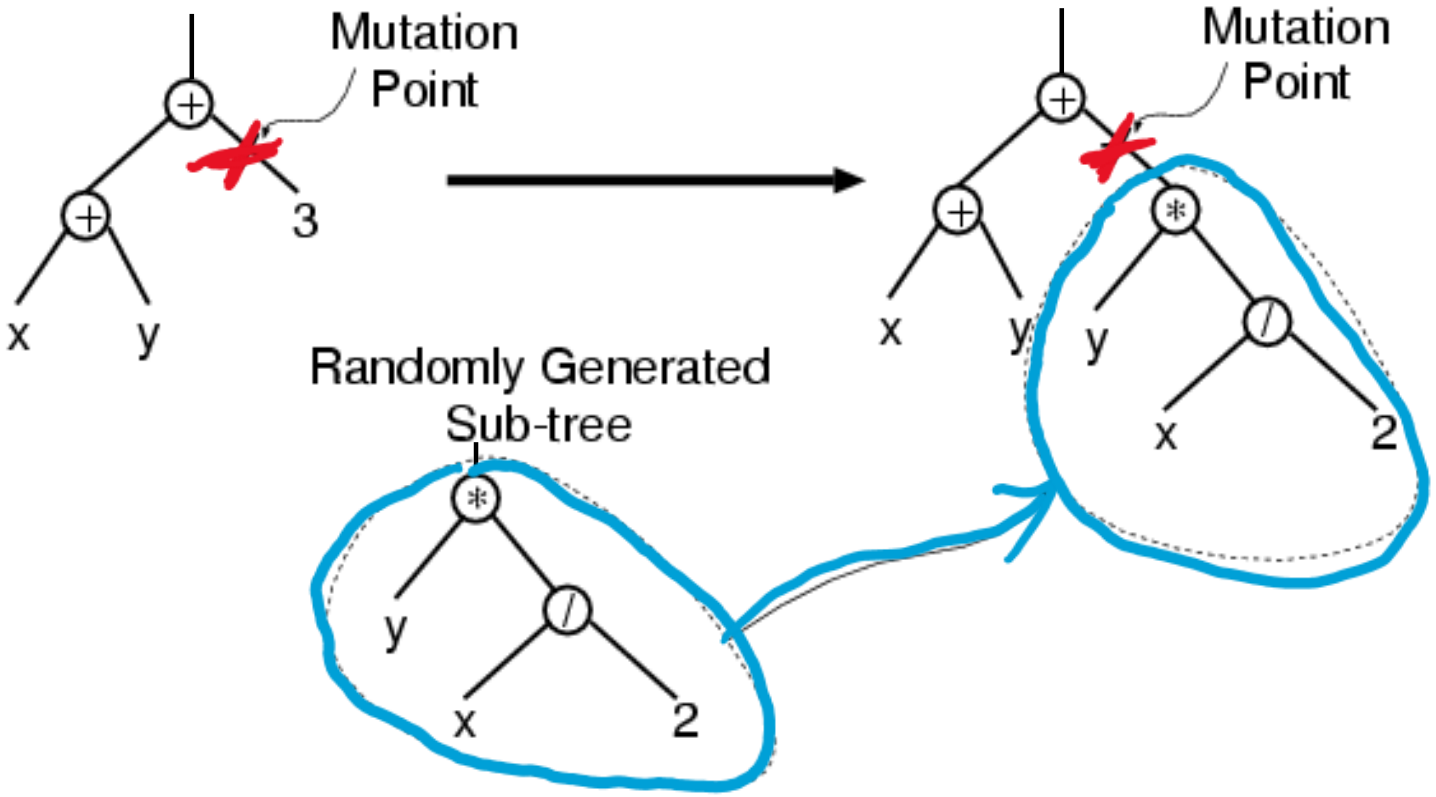
  Resulting offspring: **(x / 2) + 3**

Sub-tree Crossover (3)

- Note the root node/edge can also be selected as a crossover point!
- Some time **crossover points are not selected with uniform probability**.
  - Koza used a method where **functions** are chosen **90%** of the times, while **leaves** are selected **10%** of the times.
  - This was done to avoid *excessive exchange of very small trees* (GP trees tend to have many leaves).

Sub-tree Mutation (1)

- **Mutation** randomly selects a mutation point in a tree and substitutes the sub-tree rooted there with a **randomly generated sub-tree**.



Sub-tree Mutation (2)

- Seen as acting on *expressions*, the same mutation operation corresponds to

  Parent: **(x + y) + 3**

  Expression excised from parent 1 for deletion: **3**

  Parent 1 thus donates the structure: **(x + y) + ?**

  Randomly generated expression to be inserted in the parent (replaces ?): **y * (x / 2)**

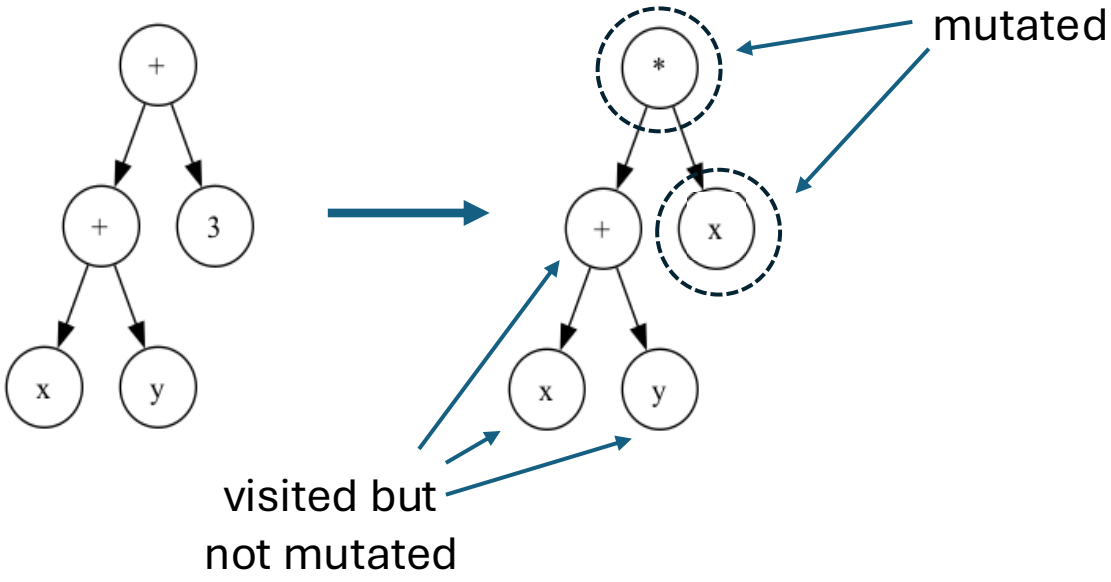
  Resulting offspring: **(x + y) + (y * (x / 2))**

Sub-tree Mutation (3)

- One mutation can modify many nodes/primitives/instructions at once.
- The root node/edge can also be selected as a mutation point!

Point Mutation

- **Point mutation** is the equivalent for GP of GA mutation.
- Each node is visited, a (biased) coin is flipped, if heads the node is replaced with a **different random primitive** of the same *number of arguments* (**arity**) taken from the primitive set.



Point Mutation

- The coin is biased, and nodes are mutated with a given <u>per-node</u> **mutation rate**.
- In addition, there is a <u>per-individual</u> **mutation rate**, as explained previously.
- So, we need to specify *two mutation rates*:
  - per-node
  - per-individual

Preparatory Steps of GP

- Users need to specify:
  1. The **terminal set** (“leaves”),
  2. The **function set** (internal nodes),
  3. The **fitness measure**,
  4. Certain **parameters** for controlling the run,
  5. The **termination criterion** and method for designating the result of the run.

Terminal Set (Step 1)

- Steps 1 and 2 specify the ingredients that are available to create the computer programs (**primitive set**).
- The **terminal set** may consist of
  - The program's **external inputs** (e.g. x, y, sensor data, ...),
  - **0-arity functions** (e.g. rand(), go_left()),
  - **Constants**, typically numerical (e.g., 0.1, 3, or random) but not only numerical.

Function Set (Step 2) (1)

- For some problems, the function set may consist of merely the **arithmetic functions** (+, -, *, /) and a **conditional operator**.
- But all sort of functions are allowed, e.g.

Kind of Primitive	*Example(s)*
Arithmetic	+, *, /

Function Set (Step 2) (2)

- For many other problems, the primitive set includes **specialized functions and terminals**.

  For example, if the goal is to program a robot to mop the floor

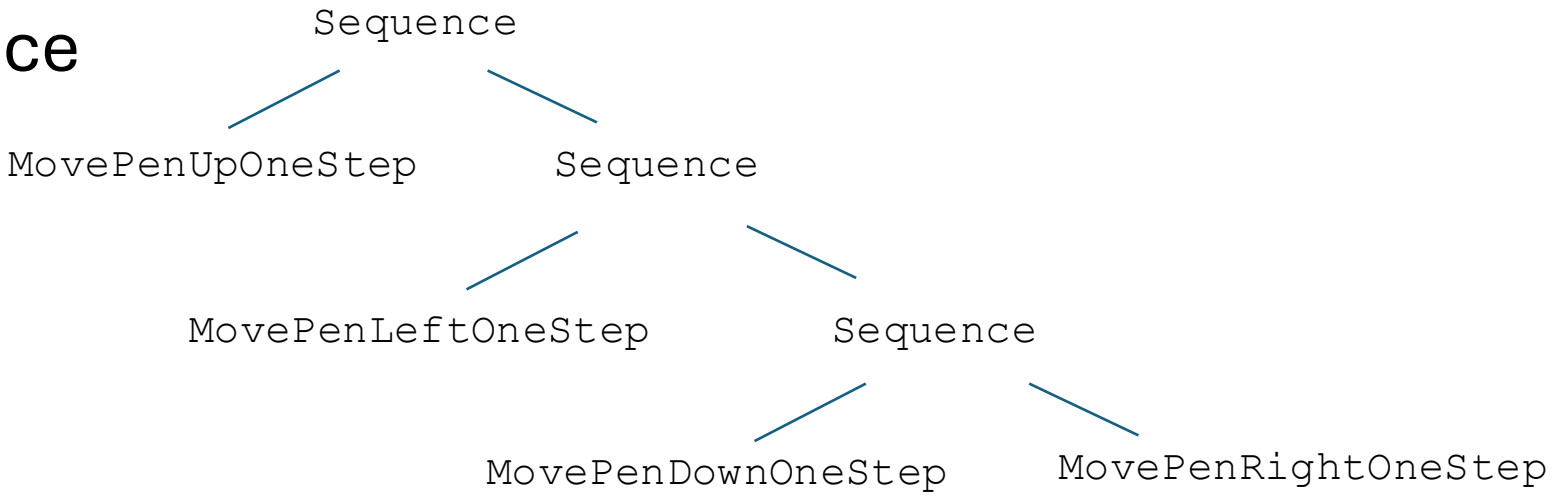
  Function set = {moving,

Programs may be solutions or recipes to build solutions (1)

- The programs evolved by GP may or may not be used as solutions, i.e., we may not be interested in transforming inputs into outputs or a program's *side effects*.
- Sometimes **programs are interpreted as a set of instructions on how to build a solution to a problem**.

Programs may be <u>solutions</u> or <u>recipes</u> to build solutions (2)

- For example, imagine you wanted to evolve a shape.
- This could be done by **creating an artificial canvas** and providing GP with **drawing primitives** and a sequencing primitives
- For instance



- When executed, this program would produce a square on the canvas.
- That is: by *evolving programs one would indirectly evolve shapes*.

Programs may be solutions or recipes to build solutions(3)

- So, GP's function sets may include functions that place elements of a solution, draw it, or grow it.
- If the goal is the automatic creation of a controller

  Function set = {integrators, differentiators, leads, lags, gains}

  Terminal set = {reference signal, plant output}
- If the goal is the synthesis of analogue electrical circuits

  Function set = {transistors, capacitors, resistors}

Fitness Measure (Step 3) (1)

- The **fitness measure** is the mechanism for giving a **high-level statement of the problem's requirements** to the GP system.
- The first two preparatory steps define the search space whereas the fitness measure **implicitly specifies the search's desired goal**.

Fitness Measure (Step 3) (2)

- Fitness can be measured in terms of
  - The amount of **error** between its output and the desired output,
  - The amount of **time** (fuel, money, etc.) required to bring a system to a desired target state,
  - The **accuracy** of the program in recognizing patterns or classifying objects into classes,
  - The **payoff** that a game-playing program produces,
  - The **compliance** of a structure with user-specified design criteria, …

Fitness Measure (Step 3) (3)

- The fitness measure is, for many practical problems, **multi-objective**, i.e. it combines two or more different elements that are often in competition with one another.
- For many problems, each program in the population is *executed* over a representative sample of different **fitness cases**.
- Fitness cases may represent different values of the program's *inputs*, different *initial conditions* of a system, different *environments*, ...

Control Parameters (Step 4)

- An important control parameter is the **population size**.
- Other control parameters include:
  - The **probabilities** of performing the genetic operations
  - The **maximum size** for programs
  - Other details of the run.

Termination Criterion (Step 5)

- We need to specify the termination criterion and the method of designating the result of the run.
- The **termination criterion** may include a **maximum number of generations** to be run as well as a **problem-specific success predicate**.
- The **best-so-far individual** is then harvested and designated as the result of the run.

Example (1)

- Goal: to automatically create a computer program whose output is equal to the values of the quadratic polynomial x^2+x+1 in the range from –1 to +1.

Example (2)

- **Step 1** – Definition of the **Terminal Set**:
  - The problem is to find a mathematical function of one independent variable, so the **terminal set must include x**.
  - In order to evolve any necessary coefficients, the terminal set also includes **numerical constants**.
  - That is: **T = {X, R}**, where R denotes random numerical terminals in some range (e.g. [–5.0,+5.0]).

Example (3)

- **Step 2** – Definition of the **Function** Set:
  - One possible choice consists of the four ordinary arithmetic functions of addition, subtraction, multiplication, and division:

$$\mathbf{F = \{+, -, *, /\}}.$$

  - To avoid run-time errors, the **division function / is protected**: it returns a value of 1 when division by 0 is attempted, otherwise it returns the quotient of its two arguments.

Example (4)

- **Step** 3 – Definition of the **Fitness Function**:
  - The fitness of a particular individual in the population must reflect **how closely** the output of an individual program comes to x^2+x+1.
  - The **fitness measure** could be defined as the **area between** the **plot** of $\mathbf{x^2+x+1}$ and the **plot** of an **individual's** mathematical expression.
  - It is rarely possible to analytically compute the value of the area. So, this is **numerically approximated** using dozens or hundreds of different values of the independent/input variable, e.g.,

$$x \in \{-1.0, -0.9, \ldots, 0.9, 1.0\}.$$

Example (5)

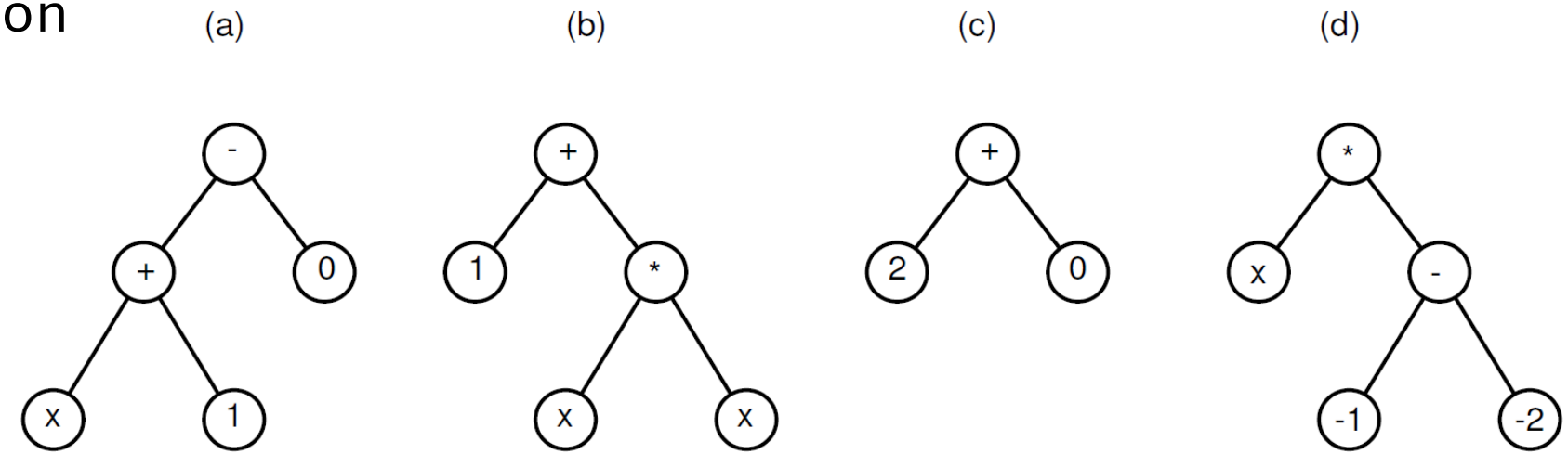
- **Step 4** – Fixing **GP Parameters**:
  - **Population size**: 4 (typically thousands or millions of individuals)
  - **Crossover probability**: 50% (traditionally high, e.g., 90%)
  - **Cloning probability**: 25% (traditionally low, e.g., 8%)
  - **Mutation probability**: 25% (traditionally very low, e.g., 1%)
  - **Architecture-altering operation probability**: 0% (if used, very low, e.g., 1%)

Example (6)

- **Step 5 – Termination Criterion**:
  - A reasonable termination criterion for this problem is that the run will continue from generation to generation until the **fitness** (**error**) of some **individual gets below 0.01**.
  - Often a **maximum number of generations** is also used as an additional stopping criterion.

Example Run (1)

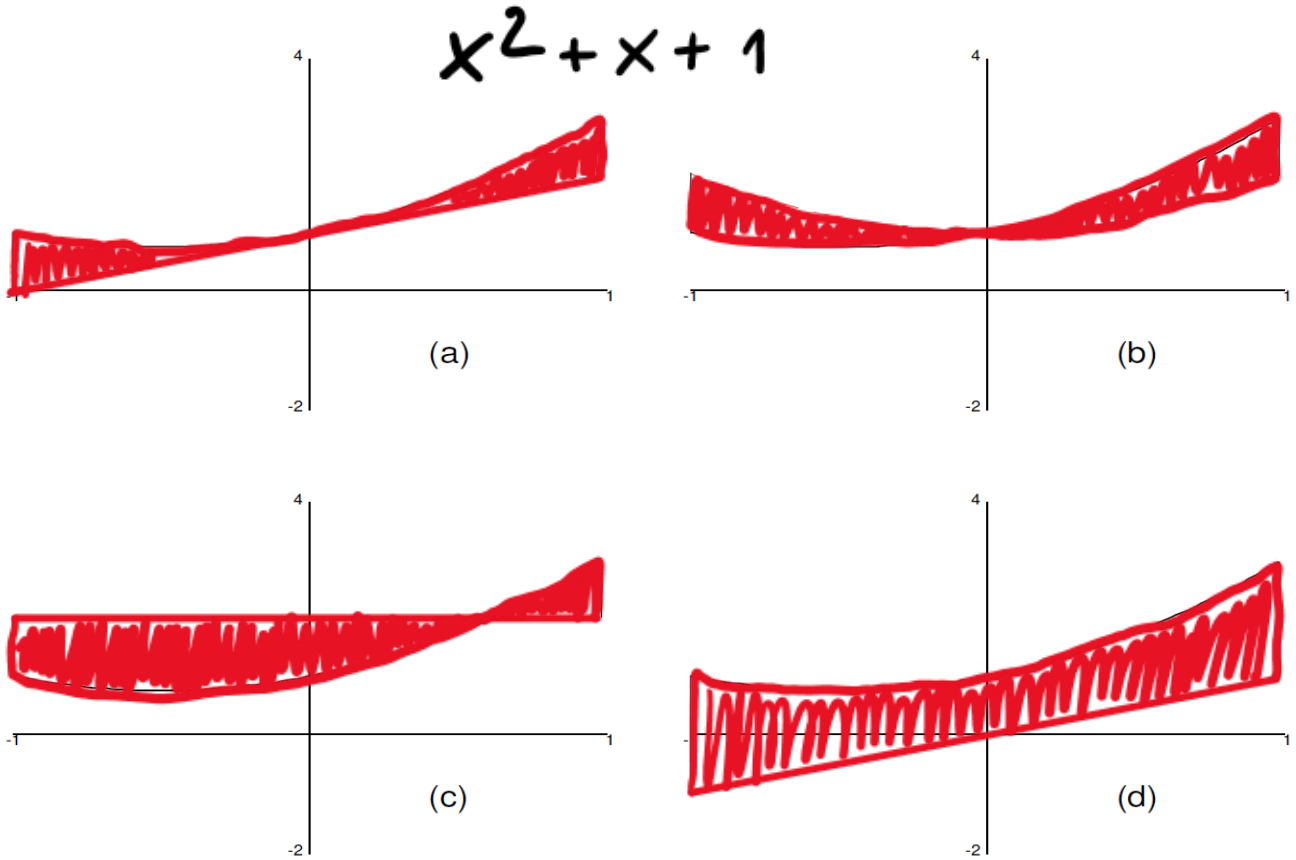
- Initial population



- Program (a) is $(x + 1) - 0 = x + 1$
- Program (b) is $1 + (x * x) = x^2 + 1$
- Program (c) is $2 + 0 = 2$
- Program (d) is $x * (-1 - -2) = x$

Example Run (2)

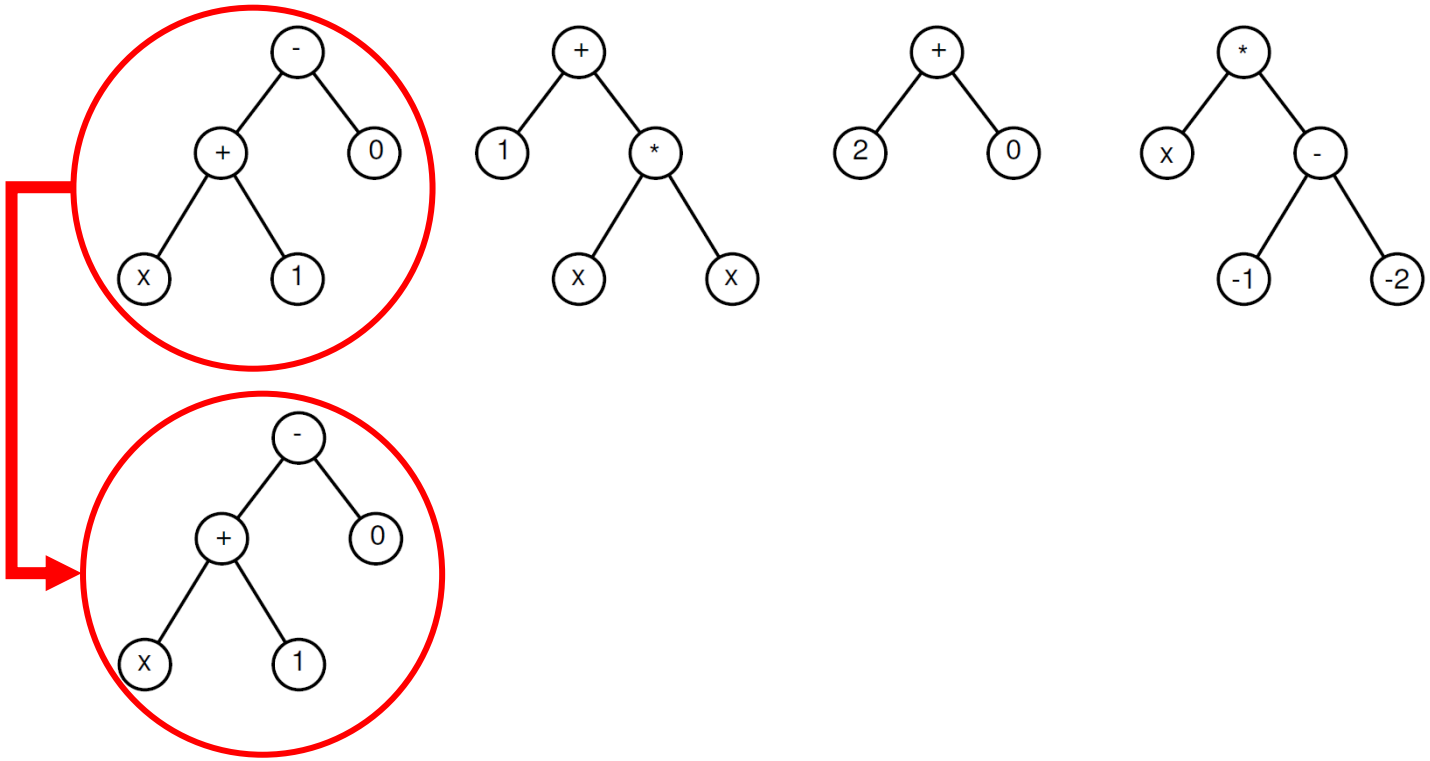
- The fitness of each of the four randomly created individuals of generation 0 is equal to the area between two curves.
- Fitness of program
  - (a): 0.67
  - (b): 1.00
  - (c): 1.67
  - (d): 2.67



Example Run (3)

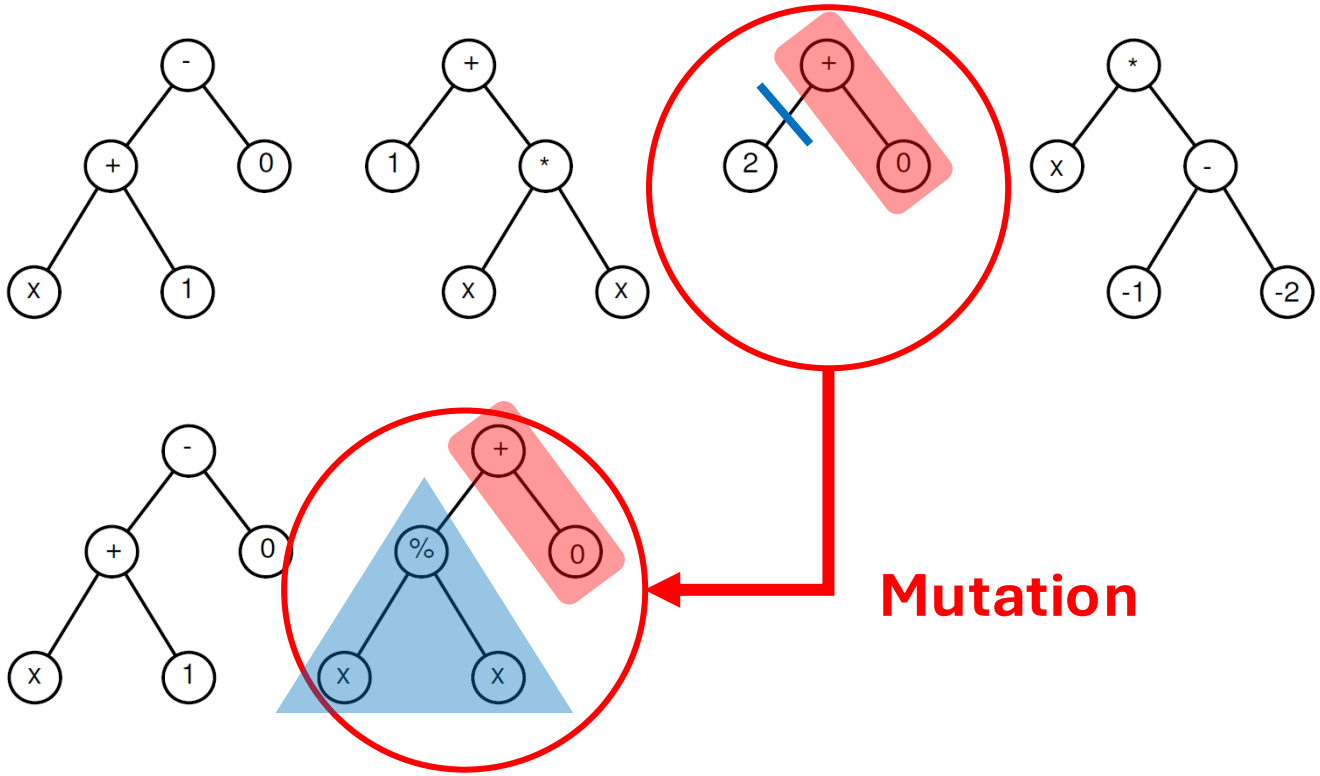
- Suppose **cloning** is chosen first, and individual a, that is (x + 1) – 0, is selected and copied unchanged into the new generation.

Cloning



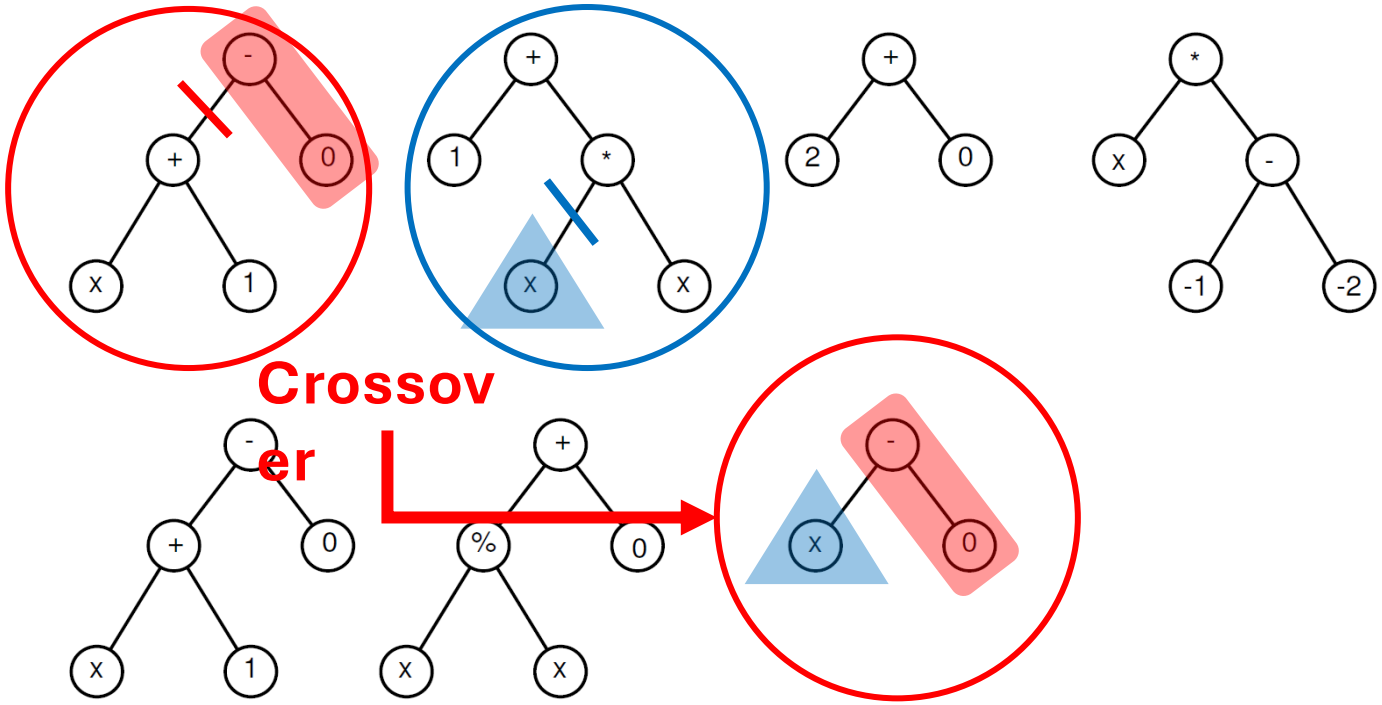
Example Run (4)

- Suppose **mutation** is picked next, individual c, that is, 2 + 0, is selected, and 2 is chosen as the mutation point; the tree (x % x) is randomly generated; it replaces 2 to produce the offspring (x % x) + 0, which is equivalent to the constant 1.



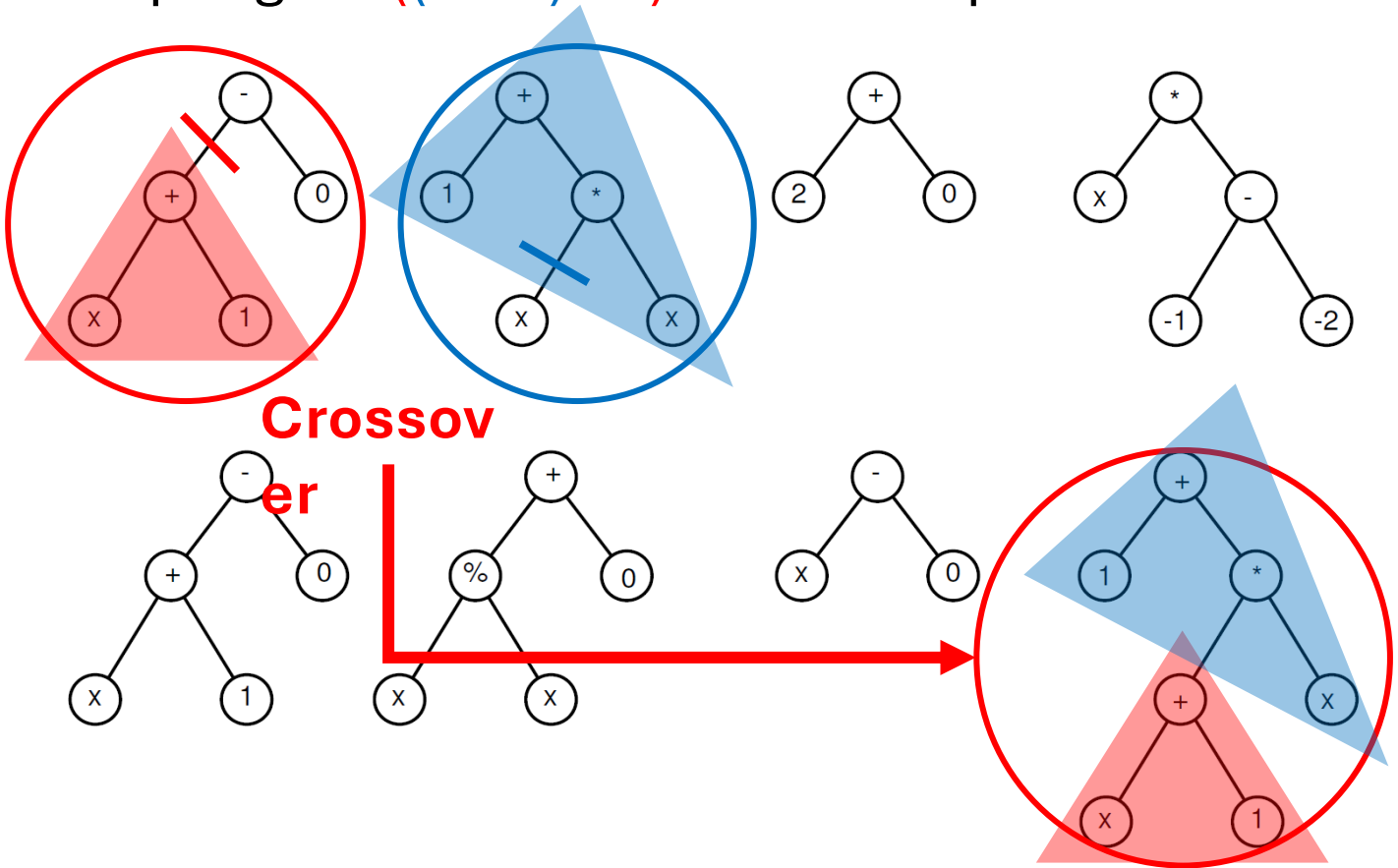
Example Run (5)

- Now **crossover** is chosen, individuals a, i.e., (x + 1) – 0, and b, i.e., 1 + (x * x), are selected; the subtree (x + 1) is excised from a; the leaf x is selected as crossover point in b and it is thus inserted in place of (x + 1) to produce the offspring x – 0, which is equivalent to **x**.



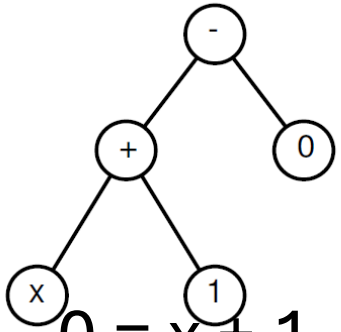
Example Run (6)

- Finally **crossover** is chosen again; individuals a, i.e., 1 + (x * x), and b, i.e., (x + 1) – 0, are selected (again); the first x in b is excised; the expression (x + 1) in a is selected as the subtree to be swapped; this is inserted in place of x to produce the offspring 1 + ((x + 1) * x) which is equivalent to $x^2 + x + 1$

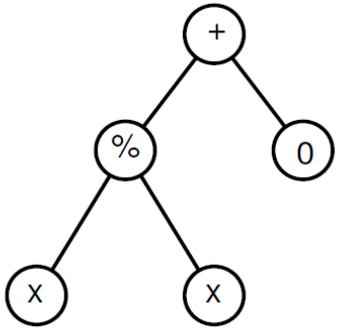


New Generation

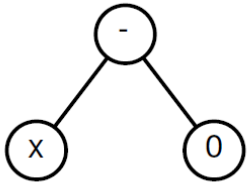
- Includes the programs:



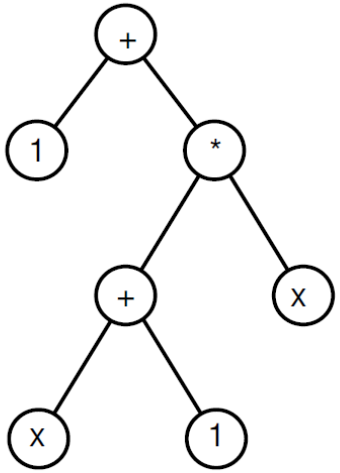
$(x + 1) - 0 = x + 1$



$(x \% x) + 0 = 1$



$x - 0 = x$



$1 + ((x + 1)*x) = x^2 + x + 1$

Fitness of the new Programs

(x + 1) – 0 has fitness 0.67 (> 0.01, don't stop)

(x % x) + 0 has fitness 1.00 (> 0.01, don't stop)

x – 0 has fitness 2.67 (> 0.01, don't stop)

1 + ((x + 1)*x) has fitness 0.00 (< 0.01, → problem solved)

More on Fitness Functions and Symbolic Regression

Fitness functions in GP (1)

- The possible applications of GP are as many as the applications for programs (virtually infinite), but one needs to **define an appropriate fitness function**.
- In problems where only the **side effects** of the program are of interest, the fitness function usually compares the **effects of the execution of a program** in some suitable environments with a desired behaviour.
- These kinds of fitness functions are very much **application dependent**.

Fitness functions in GP (2)

- On the contrary it is possible to **define general forms of fitness functions** for all the problems in which the **output of a program** is of interest.

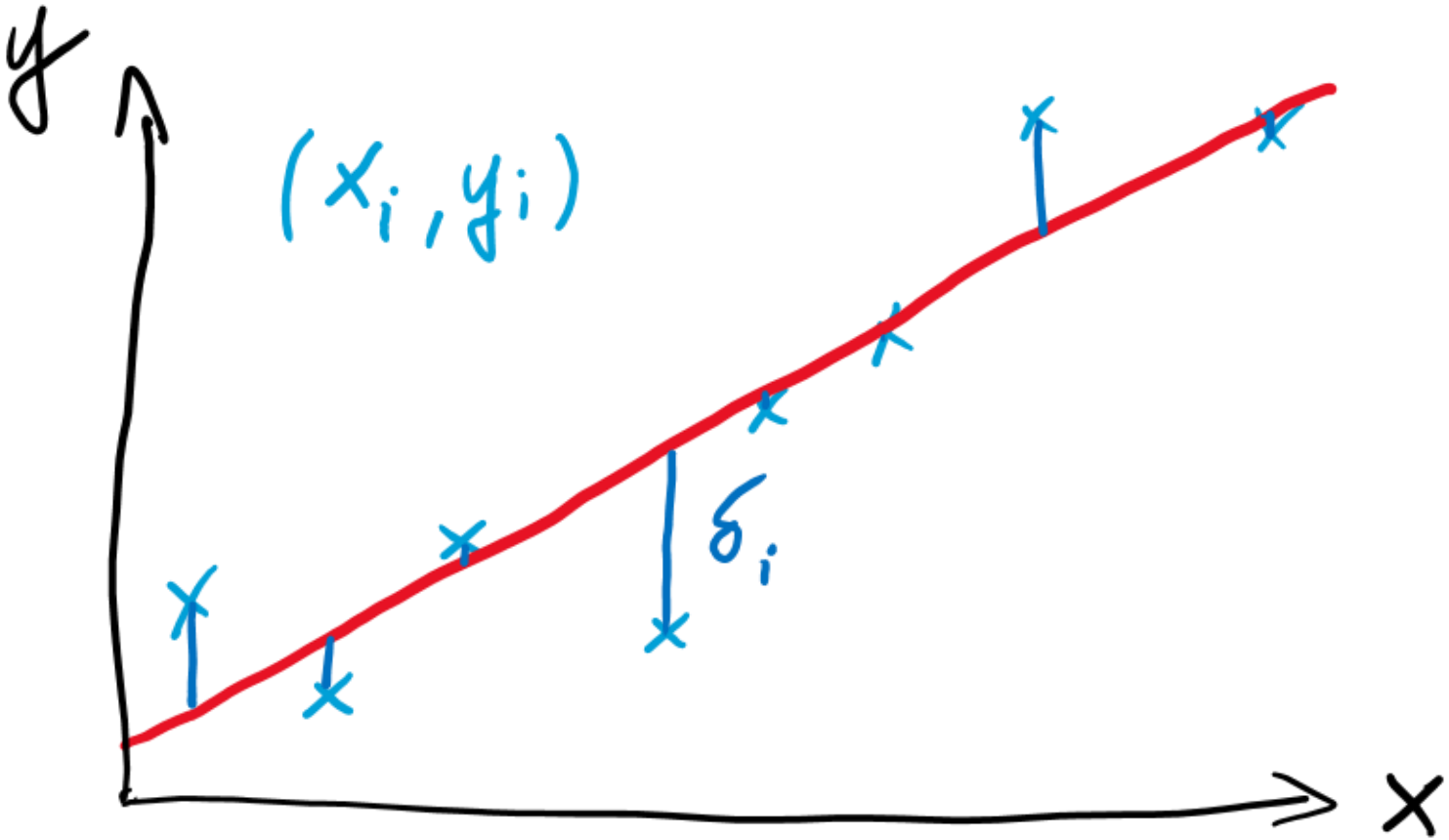
→ All of them can be seen as **symbolic regression problems**.

Symbolic Regression

- **Regression** is a technique used to interpret experimental data. It consists in finding the **coefficients of a prefixed function (model)** such that the resulting function **best fits the data**.
- If the **must good,** then the experimenter has to **try with a different function** until a good model for the data is found.
- The problem of **symbolic regression** consists in **finding a function (*with its coefficients*)** that **fits** well the data points.

REGRESSION

$$f(x_i) \overset{?}{=} y_i$$



HYPOTHESIS:

$$f(x_i) = k \cdot x_i + d$$

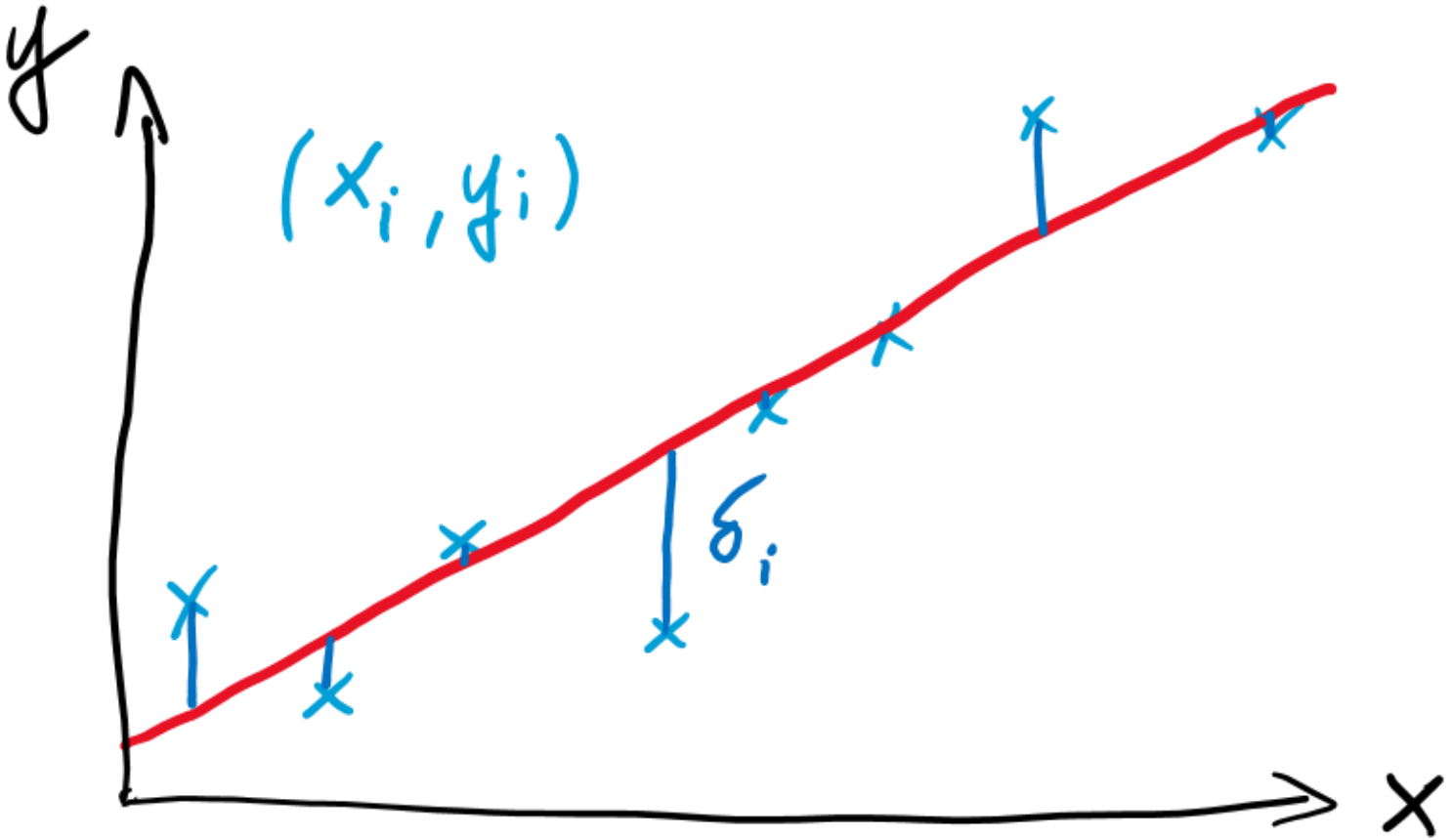
?

$$\sum_i (y_i - f(x_i))^2$$

mins

SYMBOLIC REGRESSION

$$f(x_i) \overset{?}{=} y_i$$



HYPOTHESIS:

$$f(x_i) = k \cdot x_i + d$$

? $\Rightarrow$

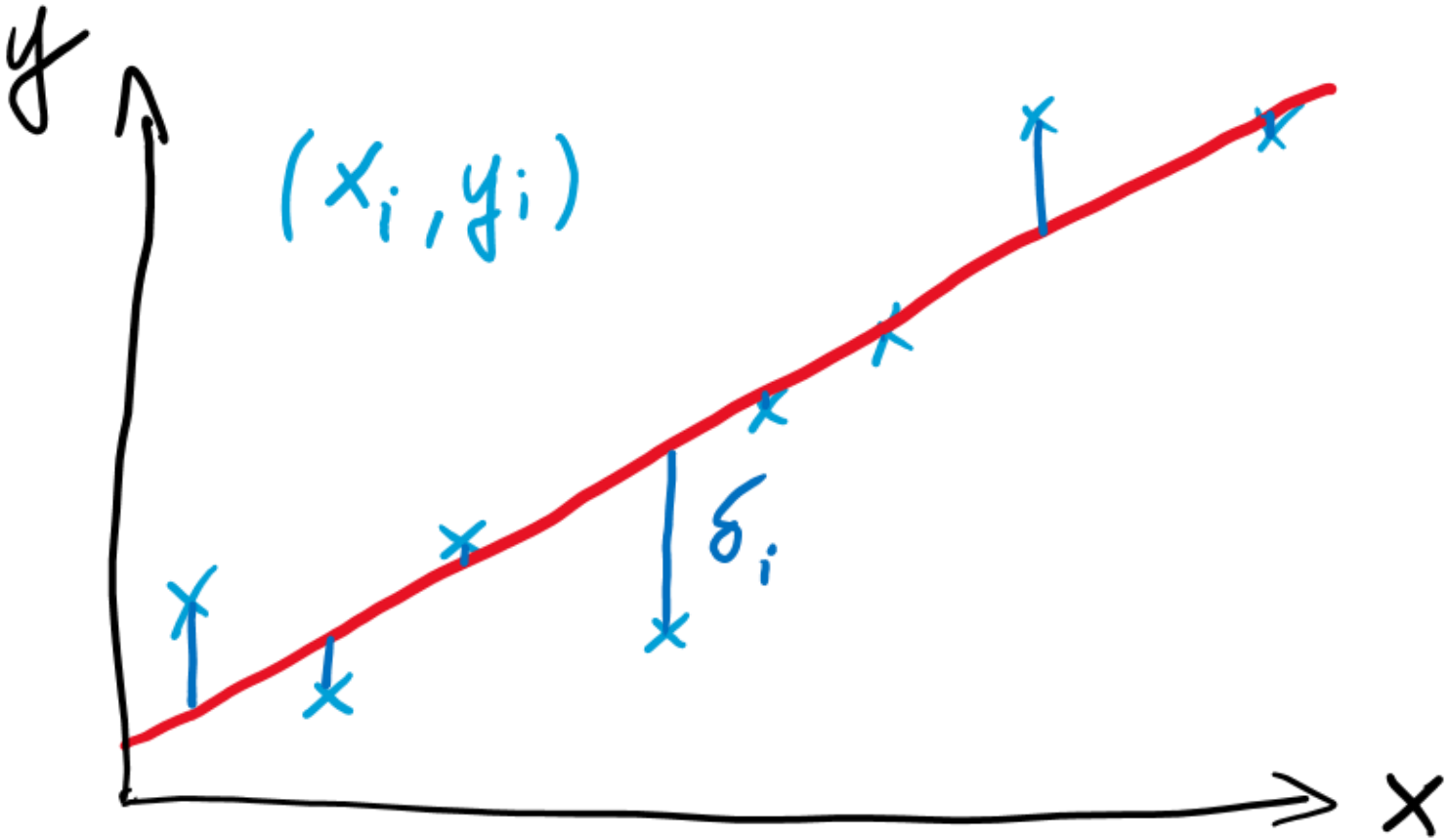
$$\sum_i (y_i - f(x_i))^2$$

mins

SYMBOLIC REGRESSION

$f(x_i) \overset{?}{=} y_i$

GP



NO MODEL HYPOTH.
MATHS OPERATORS,
ANALYTIC FUNCTIONS,
CONST. & STATES

$$\sum_i (y_i - f(x_i))^2$$

mins

Steps to Solve Symbolic Regression Problems

1. **Collect a set of data points**, where each point represents the (measured) values taken by some variables at a certain time or in a certain repetition of an experiment $(x_1, x_2, \dots x_n, y)$
2. **Select** which variable to consider as **dependent**. $y = H(x)$
3. **Define a fitness function** that measures the ability of each program to predict the value of the dependent variable given the values of the independent ones (for each data-point). $f = \Sigma (H(x) - y_i)^2$ $x = (x_1, x_2, \dots, x_n)$
4. Select an appropriate set of **functions and terminals**.
   - Terminals often include all the independent variables plus maybe others,
   - Functions are selected on the basis of knowledge on the domain.
5. Choose the remaining **GP parameters** and run GP.

Real Symbolic Regression Run

Problem: find the symbolic expression that best fits the data:

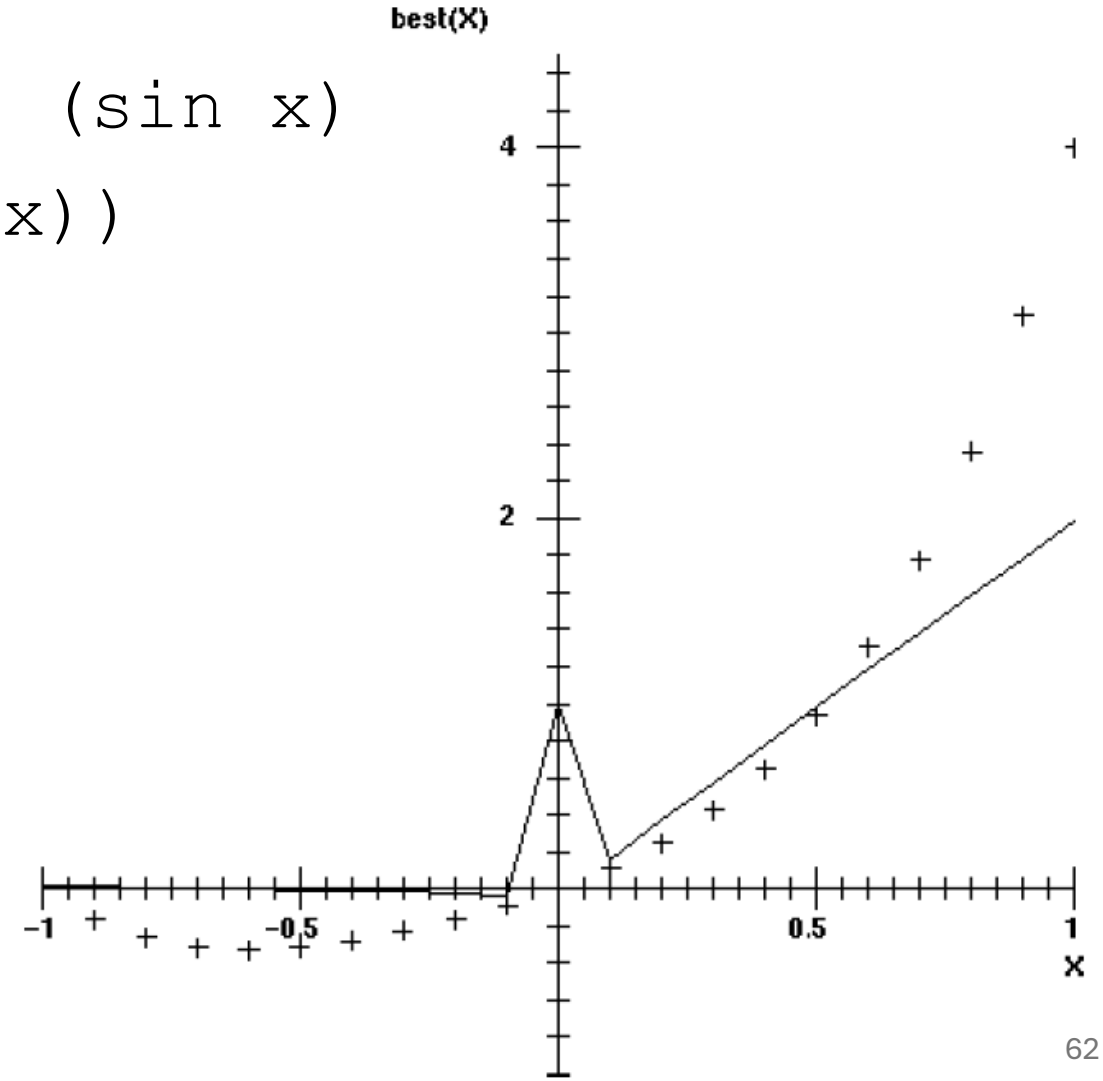
$\{(x_i, y_i)\} = \{(-1.0,0.0), (-0.9,-0.1629), (-0.8,-0.2624), \ldots, (1.0,4.0)\}$

GP parameters:

Parameter	***Value***
Population size	1000
Function set	{+ - * plog pexp sin cos pdiv}
Terminal set	{x}
Initial max depth	4
Initialisation method	Full ?
Number of generations	50
Crossover probability	0.7
Mutation probability	0
Fitness	- Sum of absolute errors

Best Program of Generation 1

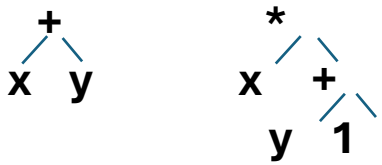
```
(+ (- (plog (pexp x)) (+ (sin x)
(- x x)))(+ (pexp (plog x))
(sin (plog x))))
```



Prefix representation of expressions:

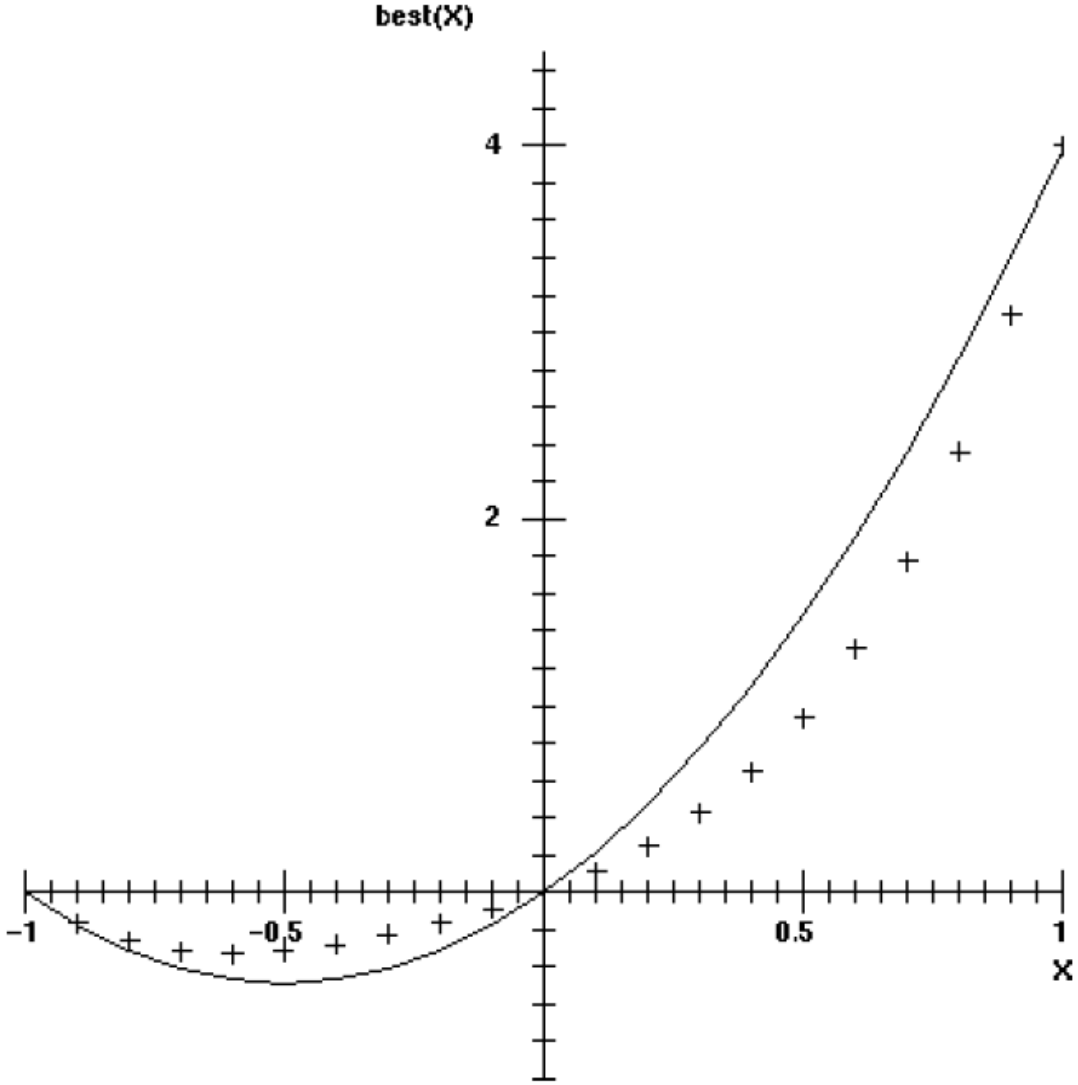
x + y ➔ (+ x y)
x * (y + 1) ➔ (* x (+ y 1))

Traverse tree in depth first order:



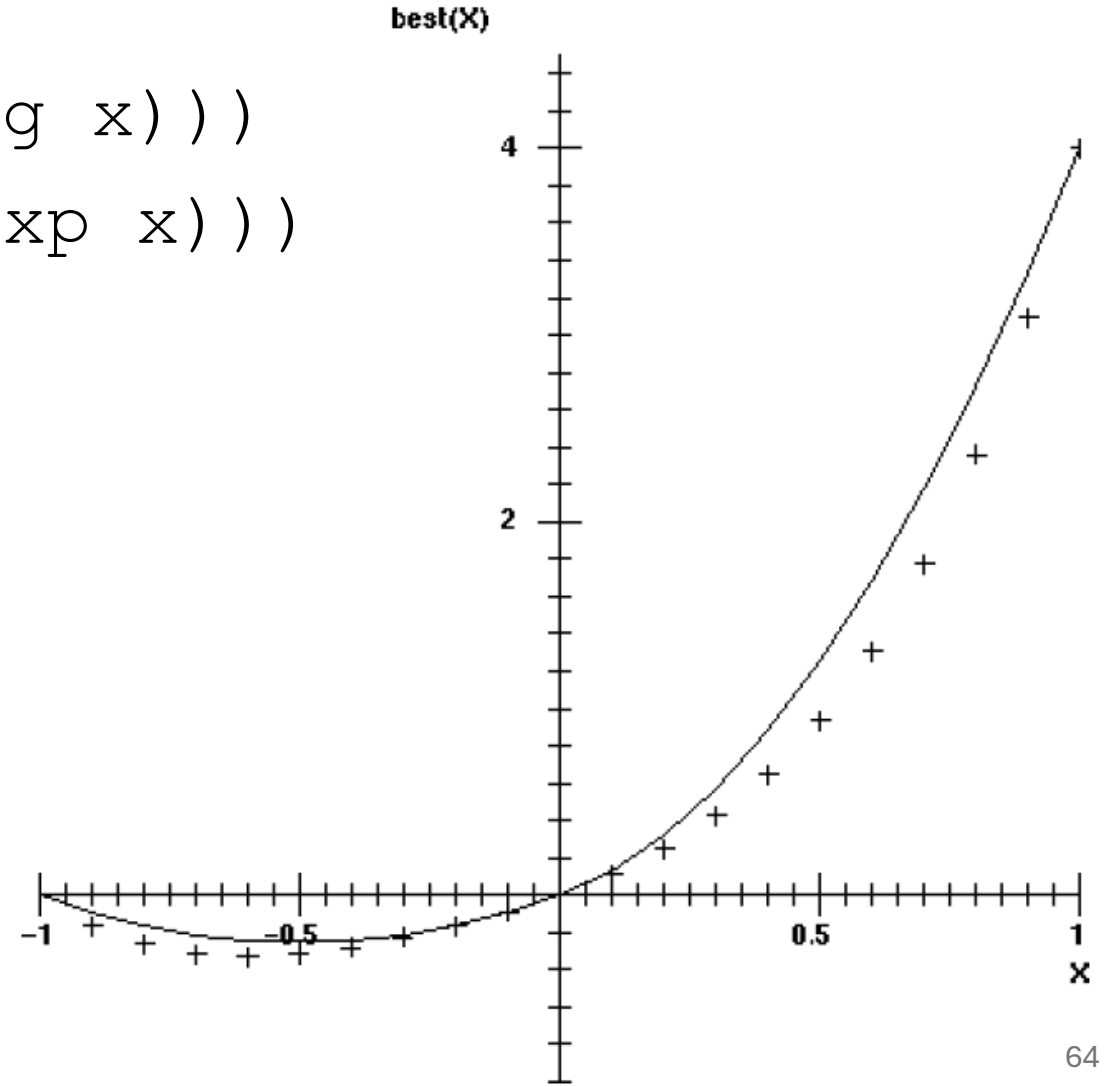
Best Program of Generation 3

```
(* (plog (- (sin x) (pex
(+ (+ x x) (cos x))))
```



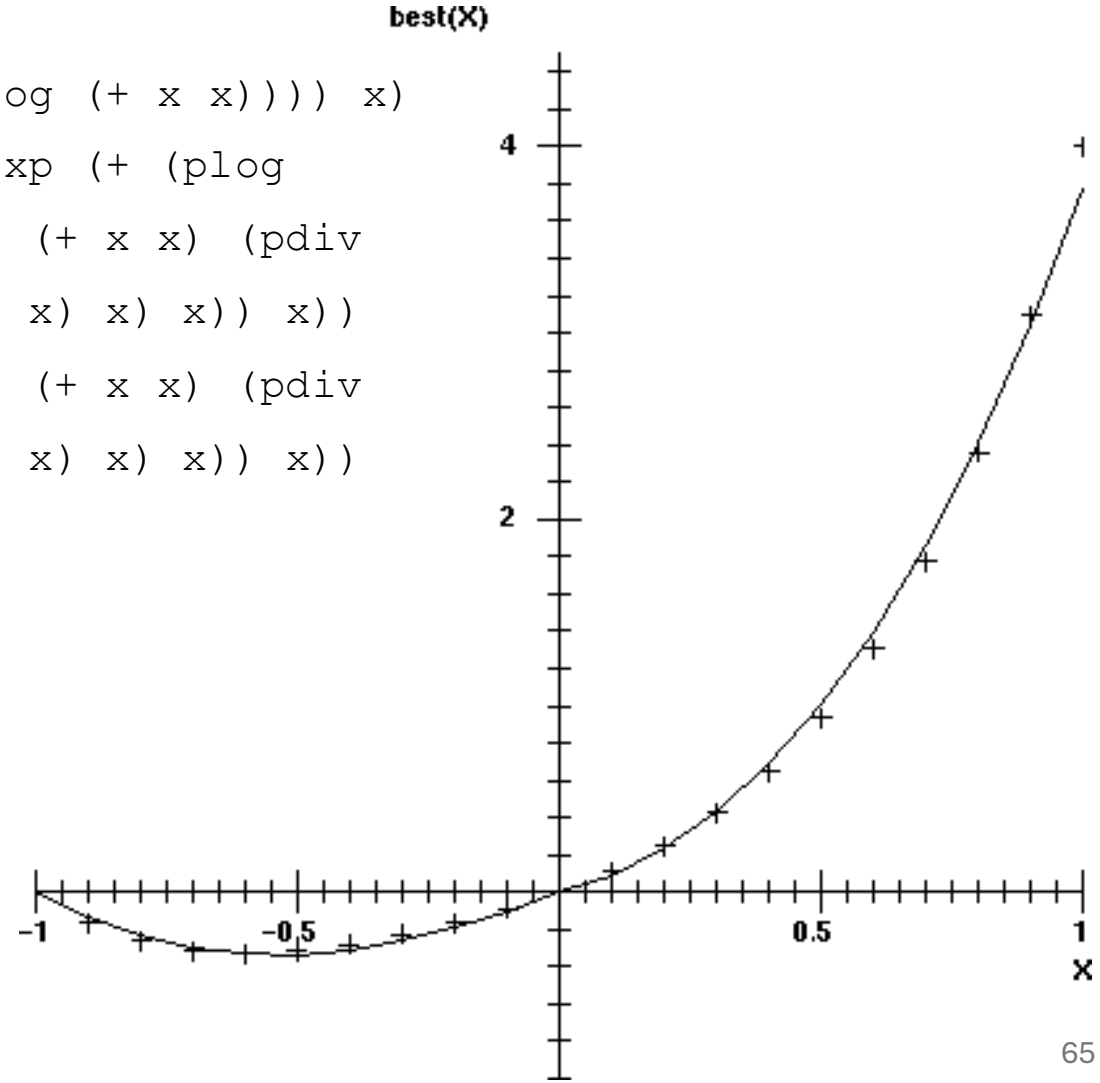
Best Program of Generation 6

```
(* (+ (+ (+ x (pexp (plog x)))
(pdiv x x)) x) (plog (pexp x)))
```



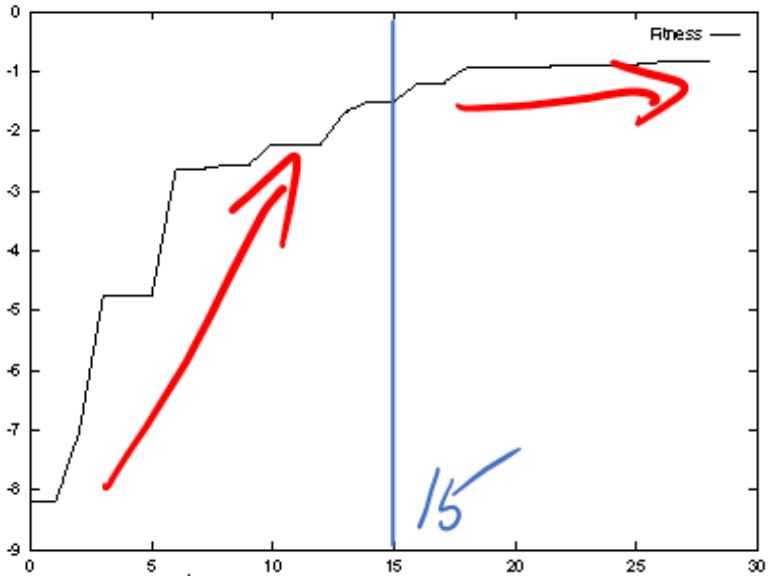
Best Program of Generation 26

```
(* (+ (+ (+ (* (+ (pexp (plog x)) (sin (plog (+ x x)))) x)
c(pexp (plog x))) (pdiv x x)) x) (plog (pexp (+ (plog
(- (sinc(+ (plog (pexp (plog (- (sin (+ (+ (+ x x) (pdiv
(* (+ (+ (+ x (pexp (plog x))) (pdiv x x)) x) x) x)) x))
(pexp x))))) x)) (pexp (plog (- (sin (+ (+ (+ x x) (pdiv
(* (+ (+ (+ x (pexp (plog x))) (pdiv x x)) x) x) x)) x))
(pexp x)))))) (plog (- x x))))))
```

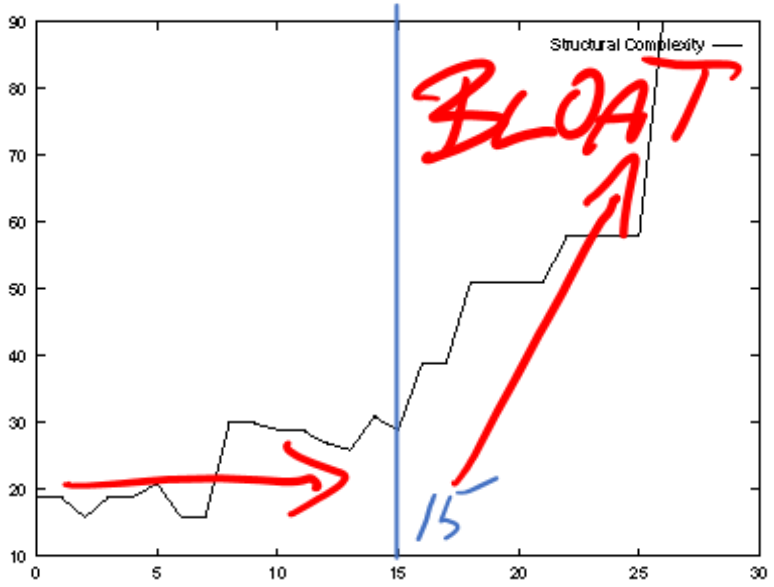


Evolution of Fitness and Size

Best-of-generation Fitness vs. Generation



Best-of-generation Size vs. Generation



Fitness increases rapidly initially, but <u>the growth slows down</u> after the first 15 generations or so

Size is more or less constant for the first 15 generations after which it starts *growing very rapidly* (**bloat**)

Another Example

- Using the symbolic regression approach GP can solve problems in completely **different domains**.
- Problem: Find a **logic circuit** that implements the XOR Boolean function using AND, OR, and NOT gates, i.e. find the Boolean function that best fits the data points

$\{(x1_i, x2_i, y_i)\} = \{(0,0,0), (0,1,1),$
$(1,0,1), (1,1,0)\}.$

$$f = \left(\sum_{i=1}^{4} \left| H(x_{1i}, x_{2i}) - y_i \right| \right)^{-1}$$

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

GP parameters

Parameter	*Value*
Population size	200
Function set	{AND OR NOT}
Terminal set	{x1 x2}
Initial max depth	3
Initialisation method	Grow ?
Number of generations	50
Crossover probability	0.9
Mutation probability	0
Fitness	1 / Sum of absolute errors

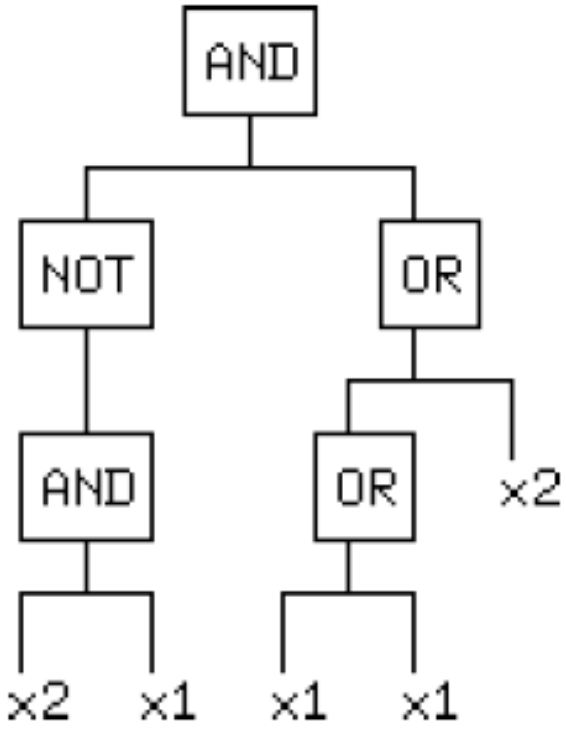
GP Run

- After 9 generations, GP found the following solution which includes 10 nodes:

```
(AND (NOT (AND x2 x1))
(OR (OR x1 x1) x2))
```

which can be interpreted as

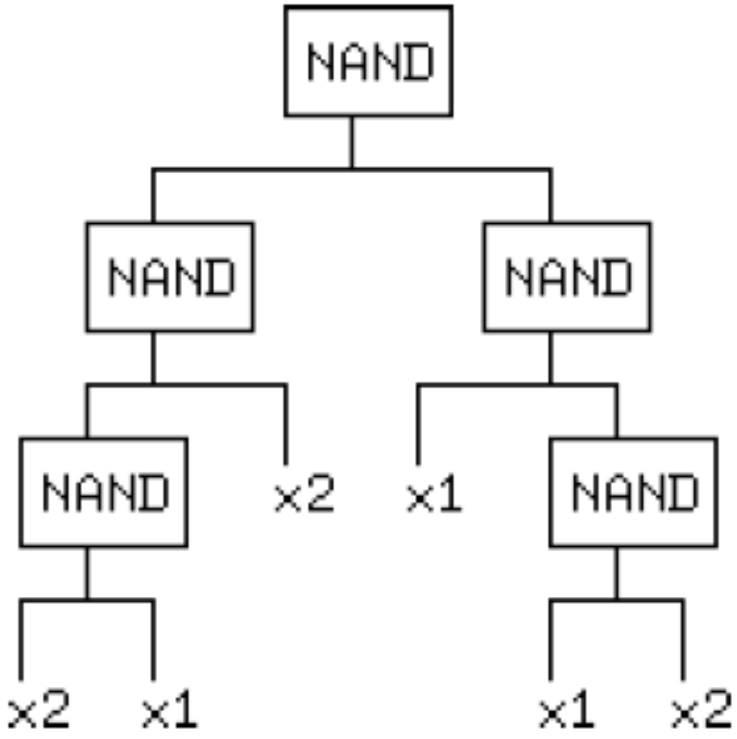
```
NOT(x1 AND x2) AND (X1 OR X2)
```



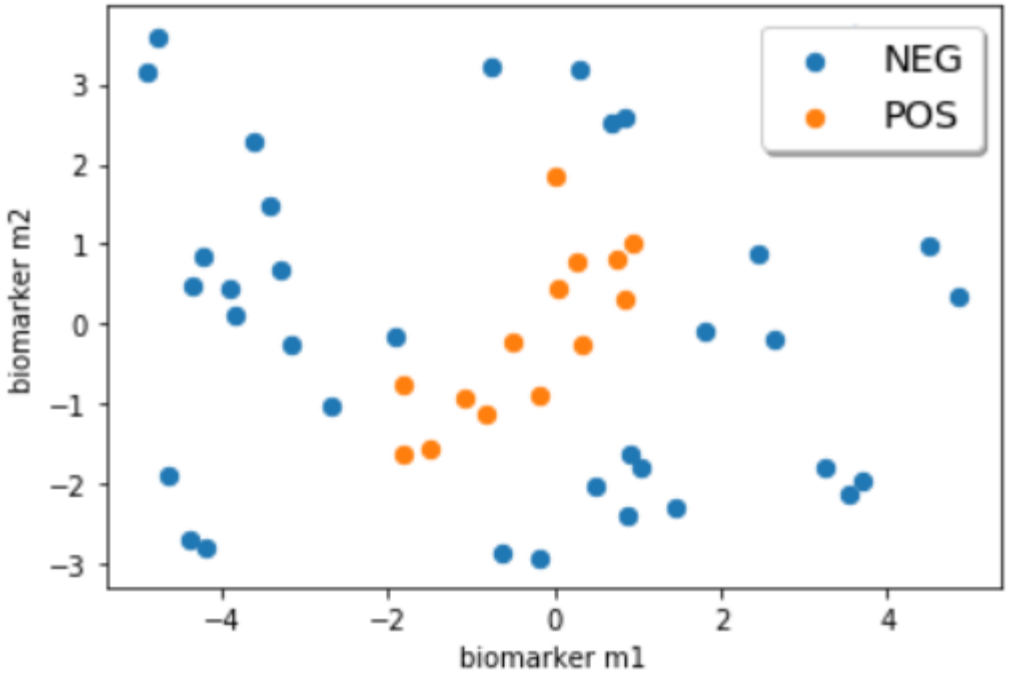
Function set: NAND

Using only NAND gates GP found the solution:

```
(NAND (NAND (NAND x2 x1) x2)
(NAND x1 (NAND x1 x2)))
```



And yet another example... Classification



$$H(m_1, m_2) = c$$

$$\sum_i \left(H(m_{1i}, m_{2i}) - c_i\right)^2$$

MINIMISE

(m_1, m_2, c)

NEG −1 POS +1

What did we learn today? (1)

- We have learned that GA, ES and GP use the same principles and concepts. One difference is that genetic operators in GP are **mutually exclusive**.
- We have learned that programs can be represented as **syntax trees**, and how to implement **sub-tree crossover**, **sub-tree mutation** and **point mutation**.
- We have learned the preparatory steps of GP. Define **terminal** and **function** set (define the search space), **fitness** measure (goal, expressed as **program output** or **action** generated), **hyper-parameters** and **termination criterion**.

What did we learn today? (2)

- Programs that the GP evolve may not be solutions, but **recipes to build solutions**
- For many problems, we need to run programs over a representative sample of different **fitness cases**
- **Termination** criterion may include **maximum number of generations** and **problem-specific success predicate**
- **Symbolic regression** can be seen as a **general form** of **fitness functions** for problems where we are interested in the **output** of a program (vs. side effects)

[CE310] Evolutionary Computation and Genetic Programming

Riccardo Poli

Next time:
Implementing GP

University of Essex
School of
Computer Science and
Electronic Engineering